

```

const User = require('../models/User');
const bcrypt = require('bcryptjs');

// @desc    Get all users
// @route    GET /api/users
exports.getAllUsers = async (req, res) => {
  try {
    const { role, department, search } = req.query;
    let query = {};

    if (role) query.role = role;
    if (department) query.department = department;
    if (search) {
      query.$or = [
        { name: { $regex: search, $options: 'i' } },
        { email: { $regex: search, $options: 'i' } },
      ];
    }

    const users = await User.find(query)
      .select('-password')
      .populate('manager', 'name email')
      .populate('teamMembers', 'name email department')
      .sort({ createdAt: -1 });
    res.json(users);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// @desc    Get single user
// @route    GET /api/users/:id
exports.getUser = async (req, res) => {
  try {
    const user = await User.findById(req.params.id)
      .select('-password')
      .populate('manager', 'name email')
      .populate('teamMembers', 'name email department');
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json(user);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// @desc    Get team members of a manager
// @route    GET /api/users/team/:managerId
exports.getTeamMembers = async (req, res) => {
  try {
    const managerId = req.params.managerId;
    const employees = await User.find({ manager: managerId, role: 'employee' })
      .select('-password')
      .sort({ name: 1 });
    res.json(employees);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// @desc    Get my team (for logged-in manager)
// @route    GET /api/users/my-team
exports.getMyTeam = async (req, res) => {

```

```

    try {
      const employees = await User.find({ manager: req.user._id, role:
'employee' })
        .select('-password')
        .sort({ name: 1 });
      res.json(employees);
    } catch (error) {
      res.status(500).json({ message: error.message });
    }
  };

// @desc    Get logged-in user's profile
// @route    GET /api/users/profile/me
exports.getMyProfile = async (req, res) => {
  try {
    const user = await User.findById(req.user._id)
      .select('-password -resetPasswordToken -resetPasswordExpires')
      .populate('manager', 'name email department');
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }
    res.json(user);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// @desc    Update logged-in user's own profile
// @route    PUT /api/users/update-profile
exports.updateMyProfile = async (req, res) => {
  try {
    const { name, email, phone, department, oldPassword, newPassword } =
req.body;

    const user = await User.findById(req.user._id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }

    if (email && email.toLowerCase() !== user.email) {
      const emailExists = await User.findOne({ email: email.toLowerCase() });
      if (emailExists) {
        return res.status(400).json({ message: 'Email already in use by another
account' });
      }
      user.email = email.toLowerCase();
    }

    if (name && name.trim()) user.name = name.trim();
    if (phone !== undefined) user.phone = phone;
    if (department !== undefined) user.department = department;

    if (newPassword) {
      if (!oldPassword) {
        return res.status(400).json({ message: 'Please provide your current
password to change it' });
      }
      const isMatch = await bcrypt.compare(oldPassword, user.password);
      if (!isMatch) {
        return res.status(400).json({ message: 'Current password is
incorrect' });
      }
      if (newPassword.length < 6) {
        return res.status(400).json({ message: 'New password must be at least 6

```

```

characters' }));
    }
    user.password = await bcrypt.hash(newPassword, 12);
  }

  await user.save();

  const updatedUser = await User.findById(user._id)
    .select('-password -resetPasswordToken -resetPasswordExpires')
    .populate('manager', 'name email department');

  res.json({
    message: 'Profile updated successfully',
    user: updatedUser,
  });
} catch (error) {
  res.status(500).json({ message: error.message });
}
};

// @desc    Create user
// @route    POST /api/users
exports.createUser = async (req, res) => {
  try {
    const { name, email, password, role, department, phone, manager } =
req.body;

    const userExists = await User.findOne({ email });
    if (userExists) {
      return res.status(400).json({ message: 'User already exists with this
email' });
    }

    const hashedPassword = await bcrypt.hash(password, 12);

    const user = await User.create({
      name,
      email,
      password: hashedPassword,
      role,
      department,
      phone,
      manager: (role === 'employee' && manager) ? manager : null,
    });

    if (role === 'employee' && manager) {
      await User.findByIdAndUpdate(manager, {
        $addToSet: { teamMembers: user._id },
      });
    }

    const createdUser = await User.findById(user._id).select('-password');
    res.status(201).json(createdUser);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// @desc    Update user (Admin only)
// @route    PUT /api/users/:id
exports.updateUser = async (req, res) => {
  try {
    const { name, email, password, role, department, phone, manager, isActive }
= req.body;

```

```

const user = await User.findById(req.params.id);
if (!user) {
  return res.status(404).json({ message: 'User not found' });
}

if (name) user.name = name;
if (email) user.email = email;
if (role) user.role = role;
if (department !== undefined) user.department = department;
if (phone !== undefined) user.phone = phone;
if (isActive !== undefined) user.isActive = isActive;
if (password) user.password = await bcrypt.hash(password, 12);

if (role === 'employee' && manager) {
  user.manager = manager;
  await User.findByIdAndUpdate(manager, { $addToSet: { teamMembers: user._id
} });
} else if (role !== 'employee') {
  if (user.manager) {
    await User.findByIdAndUpdate(user.manager, { $pull: { teamMembers:
user._id } });
  }
  user.manager = null;
}

await user.save();

const updatedUser = await User.findById(user._id)
  .select('-password')
  .populate('manager', 'name email');
res.json(updatedUser);
} catch (error) {
  res.status(500).json({ message: error.message });
}
};

// @desc    Delete user
// @route    DELETE /api/users/:id
exports.deleteUser = async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }

    if (user.manager) {
      await User.findByIdAndUpdate(user.manager, { $pull: { teamMembers:
user._id } });
    }

    await User.findByIdAndDelete(req.params.id);

    res.json({ message: 'User deleted successfully' });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

```